МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЕТ

по лабораторной работе №9

по дисциплине «Организация процессов и программирование в среде Linux»

Тема: Обмен данными через разделяемую память

Студент гр. 3311 _____ Баймухамедов Р. Р.

Преподаватель _____ Лазарев Э. А.

Санкт-Петербург
2026

ЦЕЛЬ РАБОТЫ

Познакомиться с организацией разделяемой памяти и системными функциями, обеспечивающими обмен данными между процессами.

ЗАДАНИЕ

1. Написать три программы, которые запускаются в произвольном порядке и построчно записывают свои индивидуальные данные в один общий файл через заданный промежуток времени. Пока одна программа записывает строку, две другие не должны обращаться к файлу. Интервал записи и количество строк задаются входными параметрами каждой программы. После завершения одной программы остальные должны продолжать работу. Для синхронизации использовать общие переменные, размещенные в разделяемой памяти.

2. Откомпилировать три программы и запустить их в разных терминалах с различными входными параметрами. Проверить содержимое общего файла и корректность завершения программ.

1 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Разделяемая память позволяет нескольким независимым процессам обращаться к одной области памяти. Каждый процесс присоединяет сегмент к собственному виртуальному адресному пространству. В данной работе в сегменте находятся общая переменная lock для взаимного исключения и счетчик total для нумерации записанных строк.

Функция ftok формирует ключ по существующему файлу lab9.key и идентификатору проекта. Все три программы используют один файл и один идентификатор, поэтому получают одинаковый ключ. Вызов shmget с флагом IPC_CREAT создает сегмент, если его еще нет, либо возвращает идентификатор существующего сегмента. Новый сегмент заполняется нулями операционной системой: начальное значение lock равно 0, total

равно 0. Подключение к уже существующему сегменту не сбрасывает эти переменные.

Функция shmat присоединяет сегмент и возвращает его адрес. Ошибка проверяется сравнением с (void *)-1. Вызов shmdt отсоединяет сегмент от процесса, но не удаляет его. Функция shmctl с командой IPC_STAT позволяет получить характеристики сегмента, а с IPC_RMID — пометить его для удаления; окончательное освобождение происходит после отсоединения всех процессов.

Критическая секция — участок программы, в котором выполняется запись одной полной строки в общий файл и обновление счетчика. Значение lock = 0 означает, что секция свободна, lock = 1 — что она занята. Обычная проверка переменной и последующее присваивание не обеспечивают взаимного исключения: две программы могут одновременно увидеть 0.

В реализации для Linux/GCC захват выполняется атомарной операцией __atomic_exchange_n, которая устанавливает lock в 1 и возвращает предыдущее значение. Только процесс, получивший 0, начинает запись. Остальные повторяют попытку после короткой задержки. Освобождение выполняется через __atomic_store_n с записью 0. Режимы ACQUIRE и RELEASE задают порядок доступа к общим данным; проверка __GCC_ATOMIC_INT_LOCK_FREE требует аппаратно поддерживаемых атомарных операций над int.

Сегмент не удаляется при выходе одной из программ: другие продолжают пользоваться общей блокировкой. После штатного завершения всех трех программ сегмент удаляется отдельным запуском режима --cleanup. Это позволяет запускать программы в любом порядке и не требует одновременного присутствия всех трех участников.

2 ПРОГРАММА

2.1 Подготовка исходных файлов

Создаются общий заголовочный файл lab9_common.h и три исходных файла program1.c, program2.c, program3.c. Общая реализация вынесена в заголовочный файл, а каждая программа передает ей свой номер. В результате формируются три самостоятельных исполняемых файла, запускаемые независимо друг от друга.

2.2 Общая реализация — lab9_common.h

Структура Shared содержит блокировку и счетчик строк. Функция enter ожидает освобождения критической секции, leave освобождает ее. Каждая строка целиком записывается вызовами write, пока блокировка удерживается. Файл открывается в режиме O_APPEND, поэтому новая запись добавляется в конец. После записи программа освобождает блокировку и выдерживает свой интервал; во время задержки другие программы могут писать в файл.

```
#ifndef LAB9_COMMON_H
#define LAB9_COMMON_H
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <time.h>
#include <unistd.h>

struct Shared {
    int lock;
    unsigned long total;
};

#if __GCC_ATOMIC_INT_LOCK_FREE != 2
#error "Lock-free int atomics are required"
#endif

static int parse_positive(const char *text, unsigned int *value)
{
    char *end;
    errno = 0;
```

```
    unsigned long n = strtoul(text, &end, 10);
    if (errno || text[0] < '0' || text[0] > '9' ||
        *end != '\0' || n == 0 || n > UINT_MAX) return -1;
    *value = (unsigned int)n;
    return 0;
}

static void delay_ms(unsigned int ms)
{
    struct timespec delay = {
        .tv_sec = ms / 1000,
        .tv_nsec = (long)(ms % 1000) * 1000000L
    };
    while (nanosleep(&delay, &delay) < 0 && errno == EINTR) {}
}

static void enter(struct Shared *state)
{
    while (__atomic_exchange_n(&state->lock, 1,
                               __ATOMIC_ACQUIRE) != 0)
        delay_ms(1);
}

static void leave(struct Shared *state)
{
    __atomic_store_n(&state->lock, 0, __ATOMIC_RELEASE);
}

static int write_all(int fd, const char *data, size_t size)
{
    while (size != 0) {
        ssize_t n = write(fd, data, size);
        if (n < 0 && errno == EINTR) continue;
        if (n <= 0) return -1;
        data += n;
        size -= (size_t)n;
    }
    return 0;
}

static int cleanup(key_t key)
{
    int id = shmget(key, sizeof(struct Shared), 0600);
    if (id < 0) {
        if (errno == ENOENT) {
            puts("Shared segment does not exist.");
            return EXIT_SUCCESS;
        }
        perror("shmget");
        return EXIT_FAILURE;
    }
    struct shmid_ds info;
    if (shmctl(id, IPC_STAT, &info) < 0) {
        perror("shmctl IPC_STAT");
        return EXIT_FAILURE;
    }
    if (info.shm_nattch != 0) {
        fputs("Programs are still attached. Cleanup refused.\n",
              stderr);
        return EXIT_FAILURE;
    }
```

```
    if (shmctl(id, IPC_RMID, NULL) < 0) {
        perror("shmctl IPC_RMID");
        return EXIT_FAILURE;
    }
    printf("Removed shared segment: shmid=%d\n", id);
    return EXIT_SUCCESS;
}

static int run_writer(int number, int argc, char **argv)
{
    int cleaning = argc == 2 && strcmp(argv[1], "--cleanup") == 0;
    unsigned int interval = 0, count = 0;
    if (!cleaning && (argc != 3 ||
        parse_positive(argv[1], &interval) < 0 ||
        parse_positive(argv[2], &count) < 0)) {
        fprintf(stderr, "Usage: %s INTERVAL_MS LINE_COUNT\n"
                "       %s --cleanup\n", argv[0], argv[0]);
        return EXIT_FAILURE;
    }
    key_t key = ftok("lab9.key", '9');
    if (key == (key_t)-1) {
        perror("ftok: create lab9.key first");
        return EXIT_FAILURE;
    }
    if (cleaning) return cleanup(key);
    int id = shmget(key, sizeof(struct Shared), IPC_CREAT | 0600);
    if (id < 0) { perror("shmget"); return EXIT_FAILURE; }
    struct Shared *state = shmat(id, NULL, 0);
    if (state == (void *)-1) {
        perror("shmat");
        return EXIT_FAILURE;
    }
    int fd = open("shared_output.txt",
                  O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd < 0) {
        perror("open");
        shmdt(state);
        return EXIT_FAILURE;
    }
    printf("Program %d: pid=%ld, shmid=%d, interval=%u ms, "
           "count=%u\n", number, (long)getpid(), id, interval, count);
    fflush(stdout);
    int result = EXIT_SUCCESS;
    for (unsigned int i = 0; i < count; ++i) {
        char line[160];
        enter(state);
        unsigned long serial = state->total + 1;
        int length = snprintf(line, sizeof(line),
            "%lu: Program %d, line %u/%u, pid=%ld\n",
            serial, number, i + 1, count, (long)getpid());
        int error = length < 0 || (size_t)length >= sizeof(line);
        if (!error) error = write_all(fd, line, (size_t)length) < 0;
        if (!error) state->total = serial;
        leave(state);
        if (error) {
            fputs("Cannot write a complete line.\n", stderr);
            result = EXIT_FAILURE;
            break;
        }
        fputs(line, stdout);
        fflush(stdout);
```

```
        if (i + 1 < count) delay_ms(interval);
    }
    if (close(fd) < 0) { perror("close"); result = EXIT_FAILURE; }
    if (shmdt(state) < 0) { perror("shmdt"); result = EXIT_FAILURE; }
    printf("Program %d finished: %s\n", number,
           result == EXIT_SUCCESS ? "OK" : "ERROR");
    return result;
}
#endif
```

2.3 Первая программа — program1.c

Первая программа записывает строки с обозначением Program 1. Ее интервал и количество строк передаются аргументами командной строки.

```
#include "lab9_common.h"

int main(int argc, char **argv)
{
    return run_writer(1, argc, argv);
}
```

2.4 Вторая программа — program2.c

Вторая программа использует ту же разделяемую область и общий файл, но записывает строки с обозначением Program 2 и собственными параметрами.

```
#include "lab9_common.h"

int main(int argc, char **argv)
{
    return run_writer(2, argc, argv);
}
```

2.5 Третья программа — program3.c

Третья программа записывает строки с обозначением Program 3. Общая блокировка действует одинаково для всех трех программ и не зависит от порядка их запуска.

```
#include "lab9_common.h"

int main(int argc, char **argv)
{
    return run_writer(3, argc, argv);
}
```

3 КОМПИЛЯЦИЯ И ЗАПУСК ПРОГРАММ

3.1 Компиляция

Из трех исходных файлов формируются исполняемые файлы program1, program2 и program3.

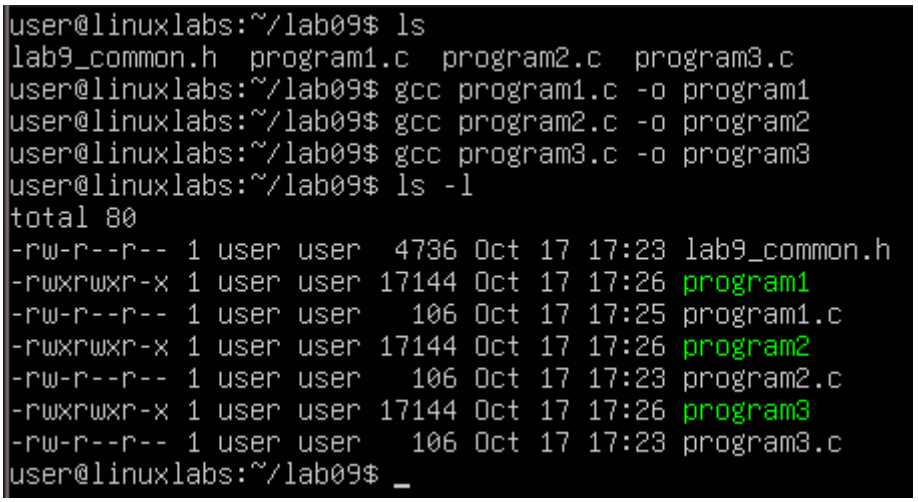


Рисунок 1 - Компиляция трех программ

3.2 Подготовка общего файла и параметров запуска

Для проверки первой программе задаются интервал 1000 мс и 3 строки, второй — 1500 мс и 5 строк, третьей — 2000 мс и 7 строк. Первую строку каждая программа записывает сразу, а интервалы выдерживаются между ее последующими записями. При близком времени запуска первая программа завершится раньше двух других.

3.3 Запуск первой программы

Первая программа записывает три строки в общий файл и завершает работу, не удаляя общий сегмент. В терминале выводятся ее PID, идентификатор сегмента, параметры запуска, записанные строки и сообщение об окончании работы.

```
user@linuxlabs:~/lab09$ ./program1 1000 3
Program 1: pid=23827, shmid=1, interval=1000 ms, count=3
1: Program 1, line 1/3, pid=23827
3: Program 1, line 2/3, pid=23827
5: Program 1, line 3/3, pid=23827
Program 1 finished: OK
user@linuxlabs:~/lab09$
```

Рисунок 2 - Работа первой программы

3.4 Запуск второй программы

Вторая программа записывает пять строк с собственным интервалом. Она использует тот же идентификатор сегмента и продолжает запись после завершения первой программы.

```
user@linuxlabs:~/lab09$ ./program2 1500 5
Program 2: pid=23839, shmid=1, interval=1500 ms, count=5
2: Program 2, line 1/5, pid=23839
6: Program 2, line 2/5, pid=23839
7: Program 2, line 3/5, pid=23839
9: Program 2, line 4/5, pid=23839
11: Program 2, line 5/5, pid=23839
Program 2 finished: OK
user@linuxlabs:~/lab09$
```

Рисунок 3 - Работа второй программы

3.5 Запуск третьей программы

Третья программа записывает семь строк и при близком времени запуска заканчивает последней. Каждая программа отсоединяет сегмент только от своего адресного пространства, поэтому окончание работы первой и второй программ не мешает третьей.

```
user@linuxlabs:~/lab09$ ./program3 2000 7
Program 3: pid=23851, shmid=1, interval=2000 ms, count=7
4: Program 3, line 1/7, pid=23851
8: Program 3, line 2/7, pid=23851
10: Program 3, line 3/7, pid=23851
12: Program 3, line 4/7, pid=23851
13: Program 3, line 5/7, pid=23851
14: Program 3, line 6/7, pid=23851
15: Program 3, line 7/7, pid=23851
Program 3 finished: OK
```

Рисунок 4 - Работа третьей программы

3.6 Проверка общего выходного файла

После завершения всех программ проверяется shared_output.txt. Для выбранных параметров в нем должно быть 15 строк: 3 от первой программы, 5 от второй и 7 от третьей. Общий счетчик формирует последовательность от 1 до 15, а внутри каждой программы сохраняется порядок ее собственных строк. Взаимное расположение записей разных программ может отличаться между запусками.

```
user@linuxlabs:~/lab09$ cat -n shared_output.txt
     1  1: Program 1, line 1/3, pid=23827
     2  2: Program 2, line 1/5, pid=23839
     3  3: Program 1, line 2/3, pid=23827
     4  4: Program 3, line 1/7, pid=23851
     5  5: Program 1, line 3/3, pid=23827
     6  6: Program 2, line 2/5, pid=23839
     7  7: Program 2, line 3/5, pid=23839
     8  8: Program 3, line 2/7, pid=23851
     9  9: Program 2, line 4/5, pid=23839
    10  10: Program 3, line 3/7, pid=23851
    11  11: Program 2, line 5/5, pid=23839
    12  12: Program 3, line 4/7, pid=23851
    13  13: Program 3, line 5/7, pid=23851
    14  14: Program 3, line 6/7, pid=23851
    15  15: Program 3, line 7/7, pid=23851
user@linuxlabs:~/lab09$
```

Рисунок 5 - Содержимое общего файла и проверка количества строк

3.7 Освобождение разделяемой памяти

После завершения опыта выполняется режим --cleanup. Он находит сегмент по тому же ключу, проверяет отсутствие подключенных процессов через IPC_STAT и удаляет его командой IPC_RMID. Очистка не выполняется автоматически при завершении отдельного писателя, чтобы не создавать разные сегменты для участников одного опыта.

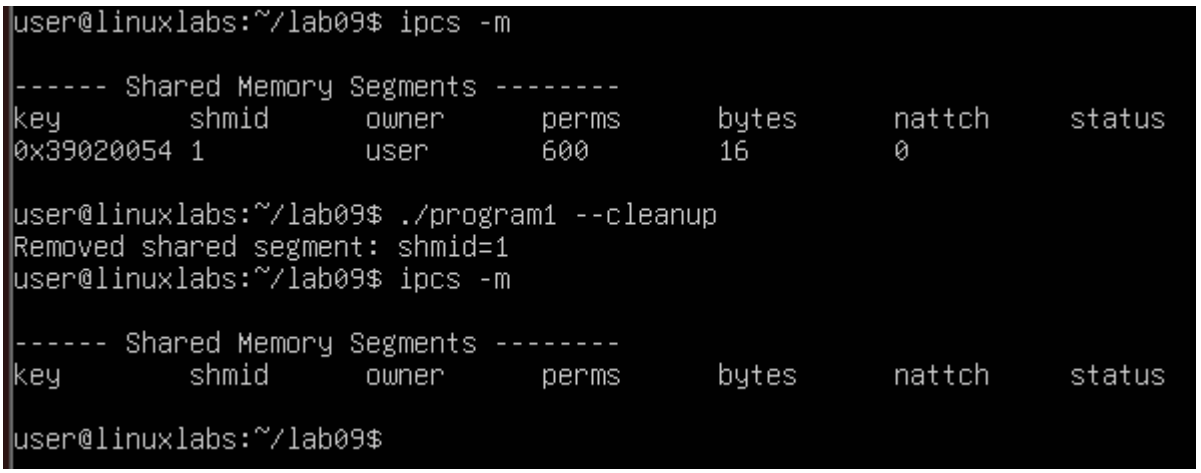


Рисунок 6 - Удаление разделяемого сегмента после завершения работы

4 АНАЛИЗ РЕЗУЛЬТАТОВ

Все три независимо запущенные программы получают доступ к одному сегменту разделяемой памяти. Совпадение shmid в их выводах подтверждает использование общего сегмента. Переменная lock обеспечивает взаимное исключение: пока одна программа записывает полную строку и обновляет total, остальные ожидают освобождения критической секции.

При параметрах 1000 мс / 3 строки, 1500 мс / 5 строк и 2000 мс / 7 строк суммарное количество записей равно 15. В общем файле сохраняются целые строки каждой программы. Общий номер записи увеличивается на единицу, а собственные номера строк каждой программы идут по порядку. Фиксированного чередования первой, второй и третьей программ не требуется.

После завершения первой программы две другие продолжают работу, поскольку блокировка освобождается после каждой строки, а разделяемый сегмент не удаляется до окончания всего опыта. Программы можно запускать в произвольном порядке: первый запущенный участник создает сегмент, последующие подключаются к нему без сброса состояния.

Отсоединение shmdt и удаление IPC_RMID выполняют разные задачи. Каждый писатель отсоединяет сегмент при завершении, а отдельная очистка после завершения всех участников освобождает ресурс ядра. Приведенная блокировка рассчитана на штатную работу: аварийное завершение процесса внутри критической секции может оставить lock равным 1.

ВЫВОД

В ходе лабораторной работы изучены создание, присоединение, отсоединение и удаление сегмента разделяемой памяти. Разработаны три независимо запускаемые программы, которые с заданными интервалами записывают индивидуальные данные в общий файл. Взаимное исключение при записи строки организовано с помощью общей переменной в разделяемой памяти; завершение одной программы не останавливает остальные.